Labo 09 — Labo pratique : la stack en un fichier

Objectif : lancer la stack complète `web → api → db` avec un seul fichier, observer l'ordre de démarrage imposé par les healthchecks, exploiter la stack au quotidien (`logs`, `exec`, `stop`, `down`), basculer entre développement et production avec des fichiers d'override, et toucher aux limites de podman-compose.

Prérequis — Labos 01 à 08 terminés. Image `api:1.0` présente, `~/labo-docker/05/Dockerfile` présent. **podman-compose installé** : `sudo apt install podman-compose` (si la version affichée par `podman compose version` est inférieure à 1.1, préférez `pipx install podman-compose`). Les sorties ci-dessous sont celles de podman-compose 1.6.0.

Fichiers fournis (`files/`) - `compose.yaml` — la stack : `db` (PostgreSQL, volume, secret, healthcheck), `api` (image ou `build: ../05`, `env_file`, `depends_on`), `web` (nginx, port 18080). - `compose.override.yaml` — ajouts de développement, lus automatiquement : base publiée sur `127.0.0.1:5432`, profil `dev`. - `compose.prod.yaml` — variante production, à combiner avec `-f` : `restart: unless-stopped`, profil `prod`. - `dot-env` — à renommer `.env` : `API_TAG=1.0`, substitué dans `compose.yaml`. - `api.env` — la configuration de l'API (labo 08). - `nginx-proxy.conf`, `index.html` — le front et le reverse proxy du labo 07.

Étape 0 — Mise en place

```
mkdir -p ~/labo-docker/09 && cd ~/labo-docker/09
cp <chemin-du-labo>/files/* . && mv dot-env .env
printf 'S3cr3t-pg' > db_password.txt
podman compose version 2>&1 | tail -2
podman compose config | grep -E '^name:|image:|context:'
```

Observez la bannière `>>>> Executing external compose provider "/usr/bin/podman-compose"` puis `podman-compose version 1.6.0` ; et dans la configuration fusionnée : `name: stack`, `image: localhost/api:1.0` (le `${API_TAG:-1.0}` a été substitué), `context: /home/<vous>/labo-docker/05`.

Explication. `podman compose` relaie vers podman-compose. `config` affiche ce que l'outil a compris du YAML, override compris — à lire avant tout `up`.

Étape 1 — `up`, et l'ordre de démarrage

```
podman compose up -d
podman compose ps
```

Observez les trois conteneurs créés dans l'ordre `stack_db_1`, `stack_api_1`, `stack_web_1`, et dans `ps` des âges décalés : `db Up 11 seconds (healthy)`, `api Up 6 seconds (healthy)`, `web Up 1 second`. Chaque service a attendu que le précédent soit `healthy`.

```
podman ps --format 'table {{.Names}}\t{{.Status}}\t{{.Ports}}'
podman network ls ; podman volume ls ; podman pod ls
```

Observez `127.0.0.1:5432->5432/tcp` sur `db` (l'override de développement a été appliqué), `0.0.0.0:18080->80/tcp` sur `web` ; un réseau `stack_default`, un volume `stack_pgdata`, et un pod `pod_stack` à `3` conteneurs **sans** infra.

Explication. Tout est préfixé par le nom du projet. Le pod est un regroupement propre à podman-compose : chaque service garde sa propre IP et son nom DNS.

Étape 2 — Exploiter la stack

```
curl -s localhost:18080/api/actuator/health ; echo ; curl -s localhost:18080/api/ ; echo
podman compose logs --tail=2 api
podman compose exec db psql -U postgres -c "CREATE TABLE clients(id serial, nom text); INSERT INTO clients(nom) VALUES ('Durand');"
podman compose exec api env | grep -E 'APP_|DB_'
podman run --rm --network stack_default alpine getent hosts api db web
```

Observez `{"status":"UP"}` et `{"message":"bonjour depuis compose","profile":"dev"}` (le message vient d'`api.env`, le profil de l'override) ; des logs préfixés `[api] |` ; `CREATE TABLE` / `INSERT 0 1` ; les variables `DB_HOST=db`, `DB_PASSWORD_FILE=/run/secrets/db_password` ; et les trois noms résolus sur `10.89.0.x`.

Explication. `podman compose exec` et `podman compose logs` ciblent un **service** par son nom ; dessous, ce sont `podman exec stack_db_1` et `podman logs stack_api_1`.

Étape 3 — Arrêter, redémarrer, et retrouver le `502`

```
podman inspect stack_api_1 --format '{{.NetworkSettings.Networks.stack_default.IPAddress}}'
podman compose stop api
curl -s -o /dev/null -w '%{http_code}\n' localhost:18080/api/actuator/health
podman compose start api ; sleep 6
podman inspect stack_api_1 --format '{{.NetworkSettings.Networks.stack_default.IPAddress}}'
curl -s -o /dev/null -w '%{http_code}\n' localhost:18080/api/actuator/health
podman compose restart web ; sleep 2
curl -s -o /dev/null -w '%{http_code}\n' localhost:18080/api/actuator/health
```

Observez `10.89.0.3`, puis `502` (ou `504`, selon le moment) quand l'API est arrêtée, puis une **autre** adresse après `start` (`10.89.0.6`) et toujours `502` : nginx garde l'ancienne IP (labo 07). Après `restart web` : `200`.

Explication. Compose n'y change rien : un `stop` / `start` libère et réattribue l'adresse, et nginx ne re-résout qu'au démarrage.

Étape 4 — `down` garde les données, `down -v` non

```
podman compose down
podman ps -a --format '{{.Names}}' ; podman network ls --format '{{.Name}}' ; podman pod ls --
format '{{.Name}}' ; podman volume ls --format '{{.Name}}'
podman compose up -d ; sleep 12
podman compose exec db psql -U postgres -c "SELECT * FROM clients;"
```

Observez après `down` : aucun conteneur, seul le réseau `podman`, aucun pod — mais `stack_pgdata` **toujours là**. Après le second `up`, la table `clients` et `Durand` sont de retour.

Explication. `down` retire ce qui est jetable ; les volumes, qui portent les données, exigent `-v`. C'est le filet de sécurité de Compose.

Étape 5 — Développement et production

```
podman compose down
podman compose -f compose.yaml -f compose.prod.yaml config | grep -E 'restart|APP_PROFILE|5432'
podman compose -f compose.yaml -f compose.prod.yaml up -d ; sleep 12
podman compose ps
podman inspect stack_api_1 --format '{{.HostConfig.RestartPolicy.Name}}'
podman exec stack_api_1 env | grep APP_PROFILE
podman compose exec api env | grep APP_PROFILE
podman compose -f compose.yaml -f compose.prod.yaml down
```

Observez dans `config` : trois `restart: unless-stopped`, `APP_PROFILE: prod`, et **aucun** `5432` (l'override n'est plus lu) ; dans `ps`, `db` sans port publié ; `unless-stopped` ; `APP_PROFILE=prod` par `podman exec`… mais `APP_PROFILE=dev` par `podman compose exec` sans `-f` : il a relu l'override.

Explication. `-f` est explicite et exclusif ; `exec` recompose l'environnement d'après les fichiers qu'on lui donne, pas d'après le conteneur. Donnez toujours les mêmes `-f` (ou `COMPOSE_FILE`), ou utilisez `podman exec`.

```
API_TAG=2.0 podman compose config | grep 'image: localhost'
```

Observez `image: localhost/api:2.0` : la substitution depuis le shell prime sur `.env`. Aucune image `2.0` n'existe — `config` ne vérifie pas.

Étape 6 — Deux API, et la limite de podman-compose

```
podman compose up -d --scale api=2 ; sleep 12
podman compose ps
podman run --rm --network stack_default alpine nslookup api 10.89.0.1 | grep Address
for i in 1 2 3 4 ; do curl -s -o /dev/null -w '%{http_code} '
localhost:18080/api/actuator/health ; done ; echo
```

Observez `stack_api_1` et `stack_api_2`, tous deux `(healthy)` ; `nslookup` renvoie, après la ligne du serveur, **deux** adresses pour `api` (`10.89.0.3` et `10.89.0.4`) ; quatre `200`.

```
podman compose down -v 2>&1 | tail -2
podman ps -a --format '{{.Names}}'
podman rm -f -t 0 stack_api_2
podman compose down -v
podman ps -a --format '{{.Names}}' ; podman pod ls --format '{{.Name}}' ; podman volume ls --
format '{{.Name}}'
```

Observez l'échec du premier `down -v` : `cannot remove container … as it is running` et `"stack_default" has associated containers` — `stack_api_2` est resté, et `stack_db_1` avec lui. Après suppression nommée de `stack_api_2`, le second `down -v` râle sur ce qui a déjà disparu (`no container with name or ID "stack_web_1" found`) mais nettoie le reste : plus de conteneur, de pod, ni de volume.

Explication. podman-compose ne gère pas les instances issues de `--scale` au `down`. Docker Compose le fait. Retenez la réparation : lire `podman ps -a`, supprimer par nom, relancer `down`.

Étape 7 — Construire depuis Compose

```
podman compose build 2>&1 | tail -2
```

Observez `Successfully tagged localhost/api:1.0` : le `build: context: ../05` a rejoué le Dockerfile du labo 05 (en cache), et l'image porte le nom déclaré par `image:`.

Explication. `build` + `image` dans le même service : Compose construit **et** nomme. En CI, `podman compose build` puis `podman compose up -d` suffisent à tester la stack entière.

Nettoyage

```
podman compose down -v
podman ps -a --format '{{.Names}}' ; podman volume ls --format '{{.Name}}' ; podman network ls
--format '{{.Name}}'
```

Observez que tout est vide (seul le réseau `podman` reste). Gardez `~/labo-docker/09/` entier, `db_password.txt` compris, et les images `api:1.0`, `postgres:16-alpine`, `nginx:alpine` : le labo 10 exploite cette stack.

Ce que vous devez pouvoir affirmer maintenant

- `podman compose up -d` crée réseau, volume, pod et conteneurs dans l'ordre des `depends_on` : `db` healthy avant `api`, `api` healthy avant `web`.
- Tout porte le préfixe du projet : `stack_db_1`, `stack_default`, `stack_pgdata`, `pod_stack`.
- `config` montre la fusion : override automatique, `-f` explicite qui l'ignore, `${API_TAG:-1.0}` substitué depuis `.env` ou le shell.
- `down` garde `stack_pgdata` et la table `clients` ; `down -v` les supprime.
- `stop` / `start` de l'API → nouvelle IP → `502` chez nginx, `200` après `restart web`.
- `podman compose exec` sans les bons `-f` ment (`dev`), `podman exec` dit vrai (`prod`).

- `--scale api=2` donne deux adresses au nom `api` ; podman-compose ne sait pas les retirer : `podman rm -f -t 0 stack_api_2` avant `down`.